

ID2221 - Final Project Delivery

FMA Group

Bertelli Alessandro

Mercurio Federico

Santarossa Margherita

aberte@kth.se

fedmer@kth.se

santar@kth.se

Scalable Analysis of NVD CVE Records (2015–2025)

16 October 2025

1. Background

CVE (Common Vulnerabilities and Exposure) is an example of a standardized collection of publicly disclosed vulnerabilities. Such datasets are constantly updated to continuously categorize newly discovered weaknesses and potential threats that are in this way catalogued and organized. In addition, external sources such as the National Vulnerability Database (NVD) enrich CVE records to add valuable information. For example: severity scores, type of vulnerability, affected products.

The volume of vulnerabilities, and consequentially the number of records of these datasets, is constantly increasing every year, requiring scalable data frameworks and techniques to extract valuable insights.

2. Overview

Publicly disclosed software and hardware vulnerabilities are catalogued in the Common Vulnerabilities and Exposures (CVE) list and enriched by the National Vulnerability Database (NVD). The rapid growth of CVE records makes manual analysis impractical. This project builds a data-intensive pipeline that ingests NVD JSON feeds for 2015–2025, cleans and integrates them, and performs exploratory and trend analyses to derive insights useful for vulnerability management. We emphasize scalability with Apache Spark for ingestion and transformation, while using Python-based plotting for communication of results.

2.1. Contributions

(1) A reproducible Spark-based ingestion and preprocessing pipeline over multi-year NVD JSON feeds; (2) derived datasets capturing yearly trends, severity distributions, and vendor/product views; (3) visual summaries (e.g., time series of CVE counts, average CVSS per year, severity mix, hardware vs. software split, word cloud of descriptions); and (4) an executable notebook serving as the main entry point for the analysis.

3. Dataset

We use the official NVD JSON data feeds covering **2015–2025**. Each CVE entry contains metadata such as descriptions, published/last-modified timestamps, CVSS metrics (v2/v3 when available), CWE classifications, affected vendors/products, and references. The raw data ($\approx$ GB scale across years) is stored in

HDFS and loaded with Spark in multi-line JSON mode. We standardize key fields (year, severity level, base scores) and normalize nested structures.

4. Methodology

3.1 Ingestion & Storage

- Upload yearly NVD JSON files to **HDFS** (e.g., `hdfs://localhost:9000/hdfs_dir/*.json`).
- Create a **SparkSession** and read feeds with ``spark.read.option("multiLine","true").json(...)`` to handle nested arrays.

3.2 Preprocessing & Normalization

- Explode nested arrays to flatten vulnerabilities, references, and affected products.
- Derive Year: ``year = year(to_date(published))`` to align analyses on publication year.
- Severity Labeling: map CVSS base scores into categorical severities (e.g., Low/Medium/High/Critical); retain numeric base scores for aggregation.
- Vendor/Product view: extract normalized vendor and product identifiers from CPE data.

3.3 Analysis

- Trend of CVEs per Year: group by ``year`` and count total CVEs.
- Severity Distribution over Time: group by ``year, severity`` and compute counts and proportions.
- Average CVSS by Year: aggregate ``avg(baseScore)`` (v3 preferred, falling back when needed).
- Hardware vs. Software Split: simple rules over the CPE part (e.g., ``a`` application, ``o`` OS, ``h`` hardware) to compare categories through time.
- Vendor/Product Concentration: top vendors/products per year by CVE count.
- Text Signals: description word cloud after tokenization/stop-word removal.

3.4 Visualization & Export

- Convert Spark aggregates to **Pandas** DataFrames for plotting.
- Generate line plots for ``CVEs per year`` and ``Average CVSS per year``; stacked/clustered bars for ``severity distribution``; comparative bars/lines for ``hardware vs. software``; and a word cloud for descriptions.

5. Results and Discussion

A detailed description of the query, methodology, and objectives of this analysis is provided in the accompanying Jupyter notebook. In this project report, we present only a general overview and summary of the key findings and results derived from the analysis.

- **Overall Volume.** The number of published CVEs shows a clear upward trend across 2015–2025 in our aggregated counts, reflecting increasing reporting, ecosystem complexity, and improved disclosure practices.
- **Severity Mix.** The share of High/Critical severity increases in several recent years, consistent with broader concern over impactful vulnerabilities. The year-by-year stacked distribution highlights

shifts in severity mix that may reflect methodology changes (CVSS v3 adoption) and reporting practices.

- **Average CVSS.** The average base score per year is relatively stable with modest variation; notable fluctuations are often aligned with years having large-scale disclosures or improvements in scoring completeness.
- **Hardware vs. Software.** Using CPE part classification, software/application (`a`) entries dominate volumes, with OS (`o`) and hardware (`h`) categories showing smaller but non-negligible counts; their trends align with platform adoption and vendor practices.
- **Vendor/Product View.** Top vendors vary by year; concentration analyses indicate that a small set of vendors/products account for a material share of CVEs, though the long tail is significant.
- **Limitations.** CVSS availability and version coverage vary across years; mapping rules (e.g., severity thresholds) introduce discretization; vendor/product normalization depends on CPE quality; we focus on descriptive analytics rather than causal inference.

6. Conclusion

We demonstrate a scalable pipeline for multi-year NVD CVE analysis using Spark for ingestion and transformation, with downstream Python visualization. The pipeline produces interpretable trends (total CVEs, severity mix, and CVSS dynamics) and supports extensions (e.g., vendor concentration, CWE families, exploit availability). The resulting artifacts (cleaned aggregates, figures, and code) enable reproducible analysis and can be adapted to streaming updates of the NVD feeds.

6.1 Potential Future Improvements

Automate weekly refresh; integrate NVD CPE matching updates; add statistical change-point detection for trend shifts; evaluate risk-weighted metrics where available.

7. Project Setup and Instructions

To ensure the full reproducibility of our analysis, we provide a set of automated scripts to deploy the required environment on a local system. The setup creates an integrated data engineering stack with Apache Hadoop (HDFS and YARN), Apache Spark, and Jupyter Lab in a dedicated Python environment.

7.1 Method 1: Pre-configured Virtual Machine (Recommended)

The most straightforward way to replicate our work is by using a pre-configured Virtual Machine (VM). The VM should come with the project repository and all scripts ready for use. (Download [here](#) for UTM)

1. **Locate the Project Folder:** Upon starting the VM, locate the cloned project repository, which contains all necessary components.
2. **Run the Analysis:** To start, follow the instructions outlined in section 7.3 **Daily Usage and Analysis Execution**.

7.2 Method 2: Installation on a Local Ubuntu System

For users who prefer to set up the environment on their own Ubuntu system, we provide a script that automates the entire installation process and a video tutorial to follow for reproducibility.

Prerequisites

- A virtual machine running a fresh version of **Ubuntu 20.04 LTS** or **22.04 LTS**.
- **sudo** (administrator) privileges.
- An active internet connection to download packages.

One-Time Installation Steps This process downloads and configures all necessary software and should only be run once.

1. **Clone the repository** or download the scripts to your VM's home directory. (See how to use GitHub-LFS with the guide provided in the VMGuide folder)

2. **Make the installation script executable:**

```
Bash  
chmod +x install_stack.sh
```

3. **Run the script:**

```
Bash  
./install_stack.sh
```

Double check that the command "*hdfs namenode -format -nonInteractive*" is launched correctly, otherwise launch it in a new terminal.

4. **Reboot the VM** once the script completes to ensure all environment settings are applied correctly:

```
Bash  
sudo reboot
```

7.3 Daily Usage and Analysis Execution

After the one-time installation, use the following workflow to start, use, and stop the environment.

1. Start the Services Every time you start your VM, you must launch the Hadoop daemons.

```
Bash  
./start_stack.sh
```

This script starts both HDFS and YARN services. You can verify the daemons (**NameNode**, **DataNode**, **ResourceManager**, **NodeManager**) are running with the **jps** command.

2. Launch Jupyter Lab and Open the Notebook Once the services are running, you can start the analysis.

3. Activate the Python virtual environment:

Bash

```
source ~/dataenv/bin/activate
```

4. Start the Jupyter Lab server:

Bash

```
jupyter lab
```

5. Open the Notebook

Copy the URL provided in the terminal and paste it into your host machine's browser. Navigate to the project's root directory and open the **CVE.ipynb** file.

- Important! Makes sure to use the correct kernel with python3.11 and to set the proper environment variables depending on your hardware resources (arm64 or amd64)

6. Stop the Services

Before shutting down your VM, it is a best practice to cleanly stop all Hadoop services to prevent HDFS corruption.

Bash

```
./stop_stack.sh
```

8. References

1. <https://nvd.nist.gov/vuln/data-feeds>